

1_data_cleaning

April 13, 2026

1 Adaptive Guardrail Layer (AGL) - Data Cleaning Notebook

1.1 Purpose

This notebook prepares the final merged prompt dataset for downstream exploratory data analysis (EDA) and model training.

1.2 Goals

- Load the final dataset
- Inspect schema and basic quality issues
- Remove invalid or unusable rows
- Standardize text fields
- Handle duplicates carefully
- Validate labels and source metadata
- Save a cleaned dataset for EDA and modeling

1.3 Input

- `dataset.csv`

1.4 Expected columns

- `prompt`: user prompt text
- `label`: binary target label
- `source_dataset`: original dataset source

1.5 Output

- `dataset_cleaned.csv`
- summary tables describing the cleaning steps

```
[ ]: import os
import re
import unicodedata
from pathlib import Path
import pandas as pd
import numpy as np
pd.set_option("display.max_colwidth", None)
```

```

INPUT_PATH = OUTPUT_PATH = "../../data/processed/"

input_file = Path(INPUT_PATH) / "dataset_merged.csv"
if not os.path.exists(input_file):
    raise FileNotFoundError(f"Could not find input file: {input_file}")

print(f"Input file found: {input_file}")

```

1.5.1 Load the dataset

We begin by loading the final merged dataset and taking a first look at its shape, column names, and sample rows. This helps confirm that the file structure matches expectations before any cleaning begins.

```

[ ]: # load the dataset
df = pd.read_csv(input_file)

# print basic information
print("Dataset shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())

# display the first few rows
df.head()

```

1.5.2 Inspect the schema and data types

This step checks the column data types and whether the fields appear to match the intended meaning: - prompt should be text - label should be numeric or easily convertible to numeric - source_dataset should be text

This also helps identify columns that may need conversion before further cleaning.

```

[ ]: # print dataframe information
print(df.info())

# show data types only
print("\nData types:")
print(df.dtypes)

```

1.5.3 Assess missing values

Missing data is one of the first issues to resolve. For this project, rows without prompt text are not useful for model training or text-based analysis, so they will likely be removed. We also check whether labels or source dataset names are missing.

```

[ ]: # count missing values in each column
missing_counts = df.isna().sum()

```

```
# calculate percentage of missing values
missing_percent = (df.isna().mean() * 100).round(2)

missing_summary = pd.DataFrame({
    "missing_count": missing_counts,
    "missing_percent": missing_percent
})

missing_summary
```

1.5.4 Inspect target labels and source dataset values

Before cleaning further, we verify: - the distinct values in the label column - the distribution of classes - the list of source datasets included in the merged file

This confirms whether the label space is valid and whether there are any unexpected source names.

```
[ ]: # show unique label values
print("Label distribution:")
print(df["label"].value_counts(dropna=False))

print("\nSource dataset distribution:")
print(df["source_dataset"].value_counts(dropna=False))
```

1.5.5 Inspect duplicate records

Prompt datasets often contain exact duplicate rows or repeated prompts across source datasets. Duplicates can distort class balance, inflate apparent dataset size, and bias model performance. Here we measure duplicates at two levels: - full-row duplicates - duplicate prompts only

We inspect counts first before deciding what to remove.

```
[ ]: # count exact duplicate rows
full_duplicates = df.duplicated().sum()

# count duplicate prompts only
prompt_duplicates = df.duplicated(subset=["prompt"]).sum()

print("Exact duplicate rows:", full_duplicates)
print("Duplicate prompts:", prompt_duplicates)
```

1.5.6 Create a copy for cleaning

To preserve the raw dataset, all cleaning steps are applied to a copy rather than directly to the original dataframe.

```
[ ]: # create a working copy of the original dataframe
df_clean = df.copy()
```

```
print("Working copy created.")
print("Shape:", df_clean.shape)
```

1.5.7 Remove rows with missing or empty prompts

Rows without usable prompt text cannot support text classification. In this step, we: - drop rows where `prompt` is null - strip whitespace - remove rows where the prompt becomes empty after stripping

```
[ ]: # record starting row count
rows_before = len(df_clean)

# drop rows where prompt is missing
df_clean = df_clean.dropna(subset=["prompt"]).copy()

# convert prompt to string and strip leading/trailing whitespace
df_clean["prompt"] = df_clean["prompt"].astype(str).str.strip()

# remove rows with empty prompt strings
df_clean = df_clean[df_clean["prompt"] != ""].copy()

rows_after = len(df_clean)

print("Rows before removing missing/empty prompts:", rows_before)
print("Rows after removing missing/empty prompts:", rows_after)
print("Rows removed:", rows_before - rows_after)
```

1.5.8 Normalize prompt text

This step performs light text normalization to make the data more consistent without destroying semantic meaning.

We will: - normalize Unicode characters - standardize line endings - collapse repeated whitespace - preserve the original meaning of the prompt

This is intentionally conservative because aggressive cleaning can remove attack indicators that may matter for security classification.

```
[ ]: def normalize_text(text: str) -> str:
    if pd.isna(text):
        return text

    # normalize unicode characters to a consistent representation
    text = unicodedata.normalize("NFKC", text)

    # standardize line endings
    text = text.replace("\r\n", "\n").replace("\r", "\n")

    # collapse repeated whitespace but preserve single spaces
```

```

text = re.sub(r"\s+", " ", text)

# remove leading/trailing whitespace
text = text.strip()

return text

# apply normalization to the prompt column
df_clean["prompt"] = df_clean["prompt"].apply(normalize_text)

# quick sample view
df_clean[["prompt", "label", "source_dataset"]].head()

```

1.5.9 Remove exact duplicate rows

Exact duplicate rows add no new information and can overrepresent particular prompts. These are safe to remove first.

```

[ ]: # count rows before removing exact duplicates
rows_before = len(df_clean)

# remove exact duplicate rows
df_clean = df_clean.drop_duplicates().copy()

rows_after = len(df_clean)

print("Rows before removing exact duplicates:", rows_before)
print("Rows after removing exact duplicates:", rows_after)
print("Exact duplicates removed:", rows_before - rows_after)

```

1.5.10 Remove duplicate prompts

Next, we remove duplicate prompt text entries. This reduces repetition across the merged dataset.

```

[ ]: # check whether any prompt appears with more than one label
label_conflicts = (
    df_clean.groupby("prompt")["label"]
    .nunique()
    .reset_index(name="n_unique_labels")
)

conflicting_prompts = label_conflicts[label_conflicts["n_unique_labels"] > 1]

print("Prompts with conflicting labels:", len(conflicting_prompts))

# display a few examples if conflicts exist
if len(conflicting_prompts) > 0:
    sample_conflicts = conflicting_prompts["prompt"].head(5).tolist()

```

```

display(df_clean[df_clean["prompt"].isin(sample_conflicts)].
↳sort_values("prompt"))

# drop conflicting prompts
conflicting_prompt_list = conflicting_prompts["prompt"].tolist()
df_clean = df_clean[~df_clean["prompt"].isin(conflicting_prompt_list)]
print("Dataset size after removing conflicts:", df_clean.shape)

```

1.5.11 Cleaning summary table

This summary captures the most important outputs of the cleaning process so the steps can be documented clearly in the report and reproduced by teammates.

```

[ ]: # Build a compact cleaning summary
cleaning_summary = pd.DataFrame({
    "metric": [
        "original_rows",
        "cleaned_rows",
        "columns",
        "missing_prompts_remaining",
        "missing_labels_remaining",
        "exact_duplicate_rows_remaining",
        "duplicate_prompts_remaining",
        "conflicting_prompt_count",
        "label_0_count",
        "label_1_count"
    ],
    "value": [
        len(df),
        len(df_clean),
        df_clean.shape[1],
        df_clean["prompt"].isna().sum(),
        df_clean["label"].isna().sum(),
        df_clean.duplicated().sum(),
        df_clean.duplicated(subset=["prompt"]).sum(),
        len(conflicting_prompts),
        int((df_clean["label"] == 0).sum()),
        int((df_clean["label"] == 1).sum())
    ]
})

cleaning_summary

```

1.5.12 Final validation preview

As a final check, we inspect a random sample of cleaned rows to confirm that: - prompts look readable - labels are valid - source dataset names are populated

```
[ ]: # display a random sample of cleaned rows
df_clean.sample(10, random_state=42)[["prompt", "label", "source_dataset"]]
```

1.5.13 Save cleaned dataset

The cleaned dataset is saved for use in the EDA notebook and later model training notebooks.

```
[ ]: # save the cleaned dataset to CSV
output_file = Path(OUTPUT_PATH) / "dataset_cleaned.csv"
df_clean.to_csv(output_file, index=False)

# verify file creation
if output_file.exists():
    print(f"File created successfully: {output_file}")
else:
    raise FileNotFoundError(f"File was NOT created: {output_file}")
```